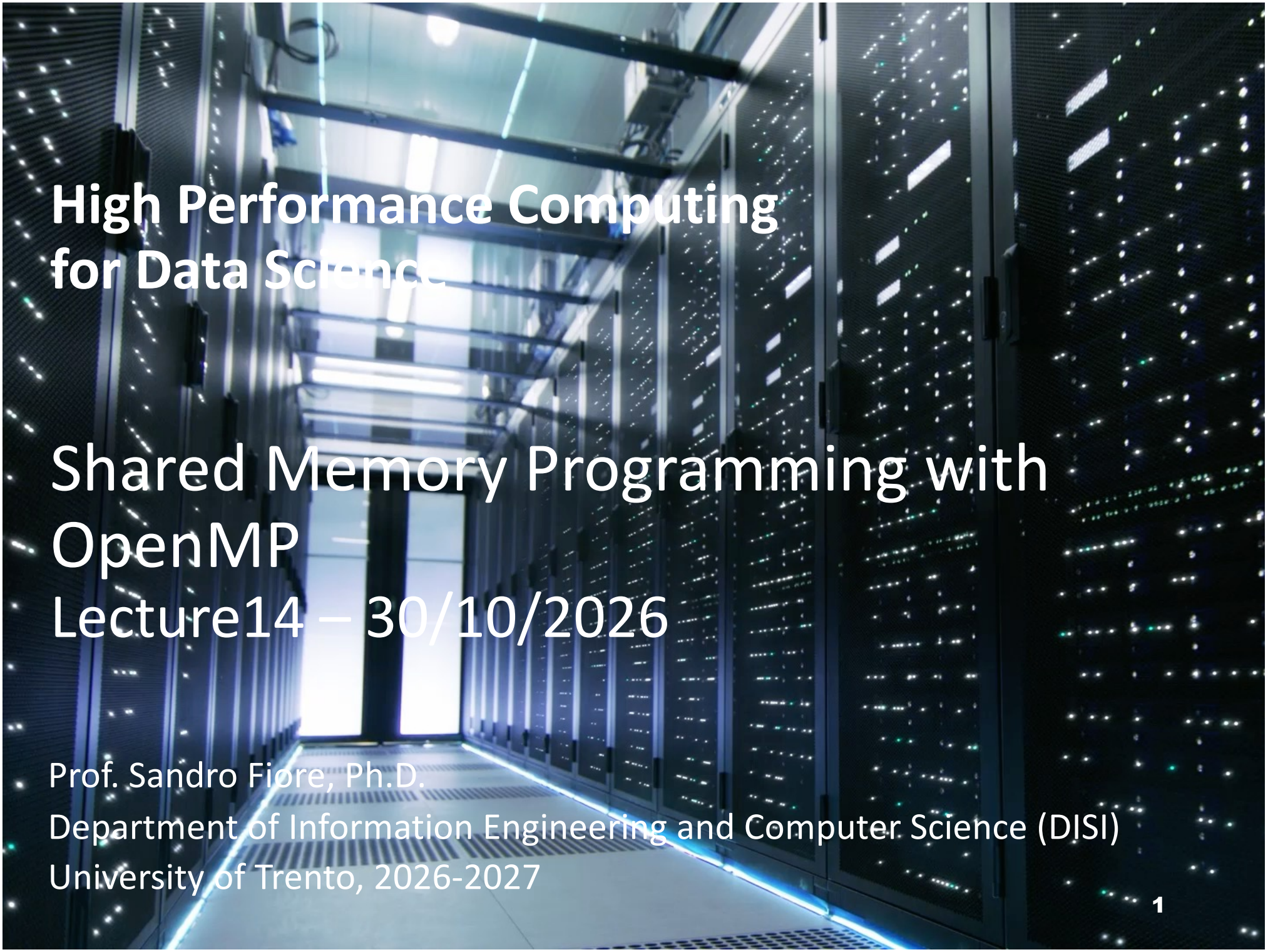


High Performance Computing for Data Science

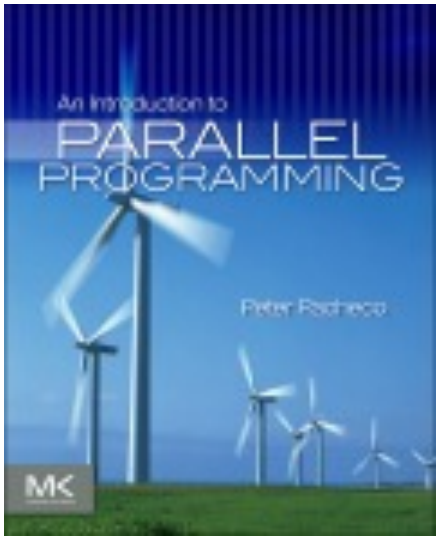
Shared Memory Programming with OpenMP

Lecture14 – 30/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Outline and material

- Recap on Lecture13
- Introduction to OpenMP
- The trapezoidal rule
- The reduction clause
- The parallel for directive



Material

Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 5
Shared Memory Programming with OpenMP

High Performance Computing for Data Science

Performance and MPI send/receive modes
Lecture 14 - 30/10/2026

Recap on Performance and MPI send and receive modes

Prof. Sandro Fiore
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Matrix-vector multiplication

a_{00}	a_{01}	$\cdots$	$a_{0,n-1}$
a_{10}	a_{11}	$\cdots$	$a_{1,n-1}$
$\vdots$	$\vdots$		$\vdots$
a_{i0}	a_{i1}	$\cdots$	$a_{i,n-1}$
$\vdots$	$\vdots$		$\vdots$
$a_{m-1,0}$	$a_{m-1,1}$	$\cdots$	$a_{m-1,n-1}$

x_0
x_1
$\vdots$
x_{n-1}

$=$

y_0
y_1
$\vdots$
$y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1}$
$\vdots$
y_{m-1}

Benchmark



comm_sz	Order of Matrix				
	1024	2048	4096	8192	16,384
1	4.1	16.0	64.0	270	1100
2	2.3	8.5	33.0	140	560
4	2.0	5.1	18.0	70	280
8	1.7	3.3	9.8	36	140
16	1.7	2.6	5.9	19	71

(Seconds)

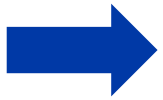
MORGAN KAUFMANN

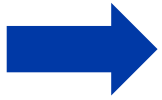
Copyright © 2010, Elsevier Inc. All rights Reserved

Speedup and efficiency

- Number of cores = p
- Serial run-time = T_{serial}
- Parallel run-time = $T_{parallel}$

$$S = \frac{T_{serial}}{T_{parallel}} \qquad E = \frac{S}{p}$$

Linear speedup  $T_{parallel} = T_{serial} / p$

Real case  $T_{parallel} = T_{serial} / p + T_{overhead}$

Copyright © 2010, Elsevier Inc. All rights Reserved

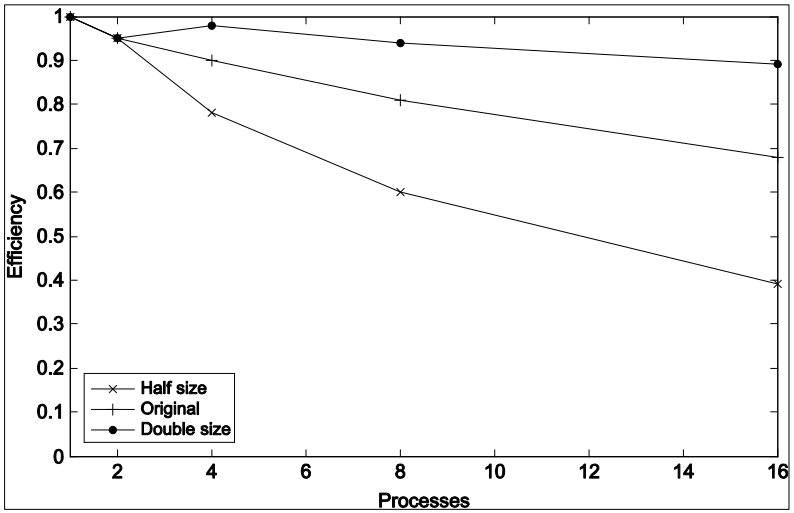
Amdahl's Law

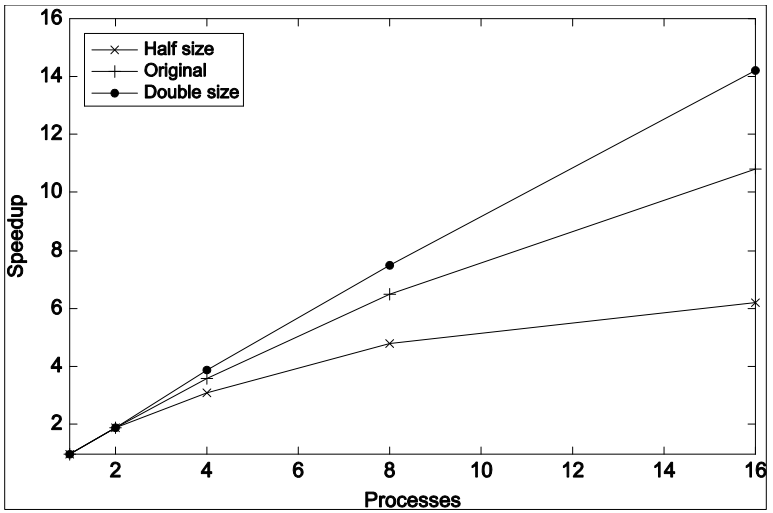
- *Unless virtually all of a serial program is parallelized, the possible speedup is going to be very limited — regardless of the number of cores available*

Copyright © 2010, Elsevier Inc. All rights Reserved

Speedups and efficiencies of parallel program on different problem sizes

	p	1	2	4	8	16
Half	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
Original	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
Double	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89





MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

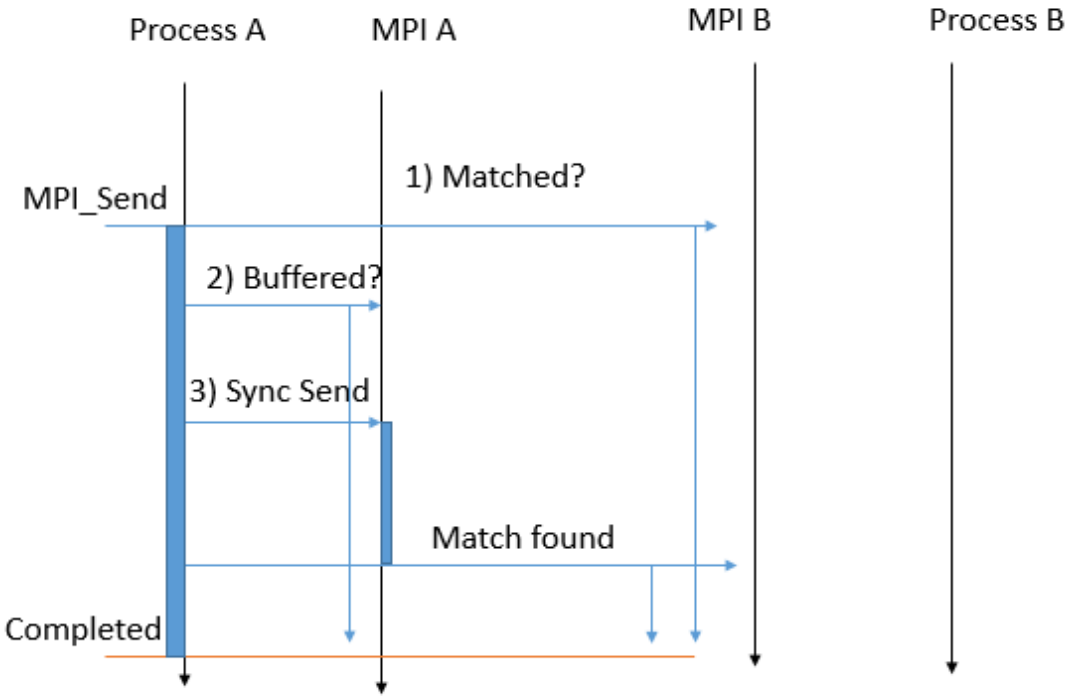
Scalability

- A program is scalable if the problem size can be increased at a rate so that the efficiency doesn't decrease as the number of processes increase.
  - Programs that can maintain a *constant efficiency* without increasing the problem size are sometimes said to be strongly scalable.
  - Programs that can maintain a *constant efficiency* if the problem size increases at the same rate as the number of processes are sometimes said to be weakly scalable.

Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Send (Standard mode)

- This is the **standard** mode
- When it is called,
  - (1) the message can be directly passed to the receive buffer
  - (2) the data is buffered (in temporary memory in the MPI implementation) or
  - (3) the function waits for a receiving process to appear.
- Therefore, it can return quickly (1)(2) or block the process for a while (3).

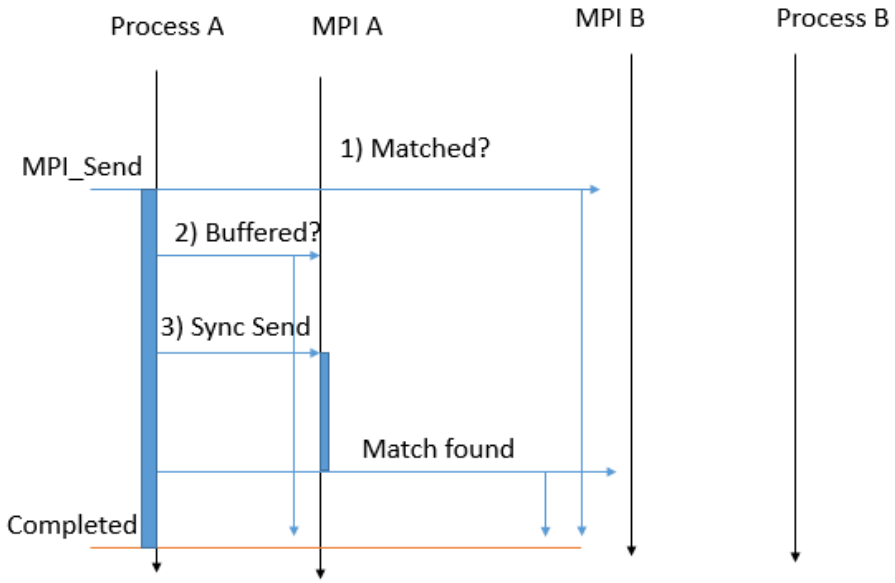


- Note: MPI decides which scenario is the best in terms of performance, memory, and so on. In any case, the data can be safely modified after the function returns.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Ssend, MPI_Bsend, MPI_Rsend (blocking functions)

- MPI_Ssend()
  - **synchronization** of source and destination
  - the behavior is predictable and safe
  - recommend for debugging purpose
- MPI_Bsend()
  - only does copy message to **buffer**
  - completes immediately
  - predictable behavior and no synchronization
  - user must allocate extra buffer space by MPI_Buffer_attach()
- MPI_Rsend()
  - completes immediately
  - will succeed only if a matching receive is already posted
  - if receiving process is not **ready**, action is undefined.
  - may improve performance



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Non blocking communication (I)

MPI_Isend, MPI_Issend, MPI_Ibsend, MPI_Irsend

- Advantages:
  - allows the separation between the **initialization** of the communication and the **completion**
  - can avoid deadlock
  - can reduce latency by posting receive calls early
- Disadvantages:
  - complex to develop, maintain and debug code

```
int MPI_Isend(void* message /* in */,
    int               count /* in */,
    MPI_Datatype      datatype /* in */,
    int               dest /* in */,
    int               tag /* in */,
    MPI_Comm          comm /* in */,
    MPI_Request*      request /* out */)

int MPI_Irecv(void* message /* out */,
    int               count /* in */,
    MPI_Datatype      datatype /* in */,
    int               source /* in */,
    int               tag /* in */,
    MPI_Comm          comm /* in */,
    MPI_Request*      request /* out */)
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Non blocking communication (II)

MPI_Isend, MPI_Issend, MPI_Ibsend, MPI_Irsend

- Non-blocking operation requires a minimum of two function calls: a call to start the operation and a call to complete the operation.
  - The "***request***" is used to query the status of the communicator or to wait for its completion.
  - The user must NOT overwrite the send buffer until the send (data transfer) is complete.
  - The user can not use the receiving buffer before the receive is complete
  - **MPI_Wait()** returns when the operation is complete, and the status is updated for a receive.
  - **MPI_Test()** returns immediately, with "flag" = true if posted operation corresponding to the "request" handle is complete.

```
int MPI_Wait(MPI_Request*     request /* in-out */,
             MPI_Status*      status /* out */)

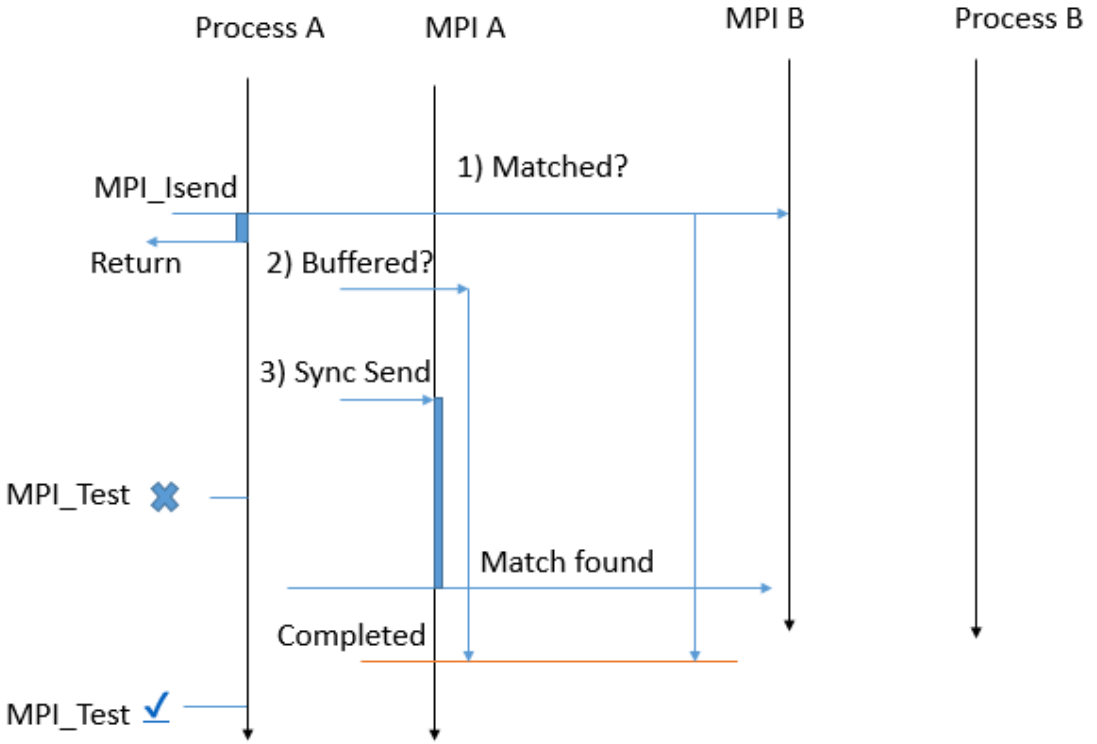
int MPI_Test(MPI_Request*     request /* out */,
             int*             flag /* out*/,
             MPI_Status*      status /* out */)
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Isend

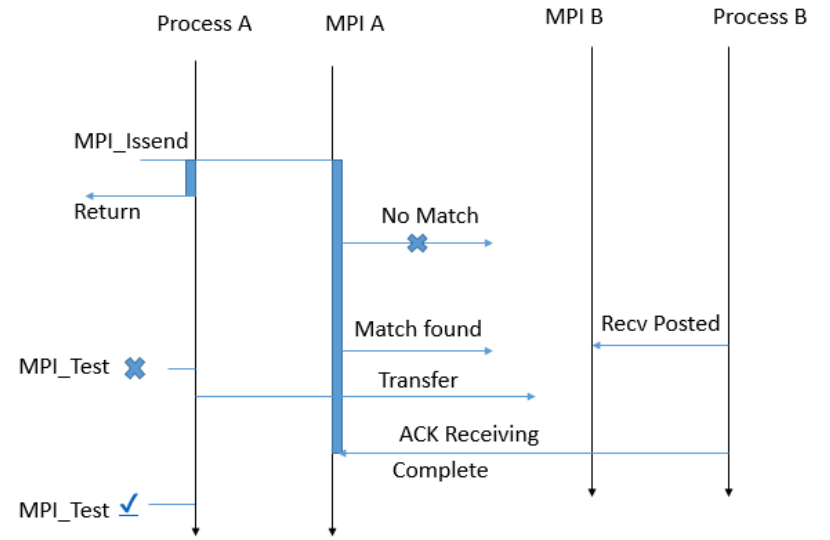
- **It is the non-blocking version of MPI_Send**
- When this function is called the function returns immediately but runs MPI_Send actions in the background of the process
- Therefore, After the function returns, the data must not be modified unless MPI_Test and MPI _Wait confirm MPI_Isend i s completed
- After the completion, the data is reusable because it either is buffered in MPI or sent to the destination

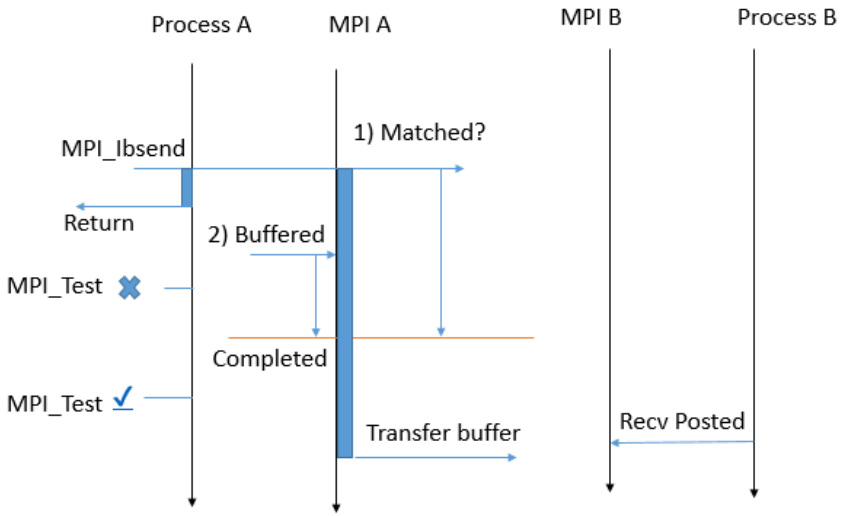


MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Issend, MPI_Ibsend, MPI_Irsend

- **MPI_Issend is the non-blocking version of MPI_Ssend.**
  - It returns immediately, but runs MPI_Ssend actions in the background.
- **MP_Ibsend (*Buffered*):**
  - This is the local non-blocking of Bsend.
  - After the test positive or wait, we can modify the source data because, if it is not sent, it is locally copied to the allocated buffer.
- **MPI_Irsend (Ready)**
  - The same as MPI_Rsend, but non-blocking.





MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Some more general & useful tips

- When there is a **deadlock**, non-blocking communication can help.
- When there is a **race condition**, blocking communication can help.
- When there is a **computationally expensive** task, a non-blocking communication, posted before the task, may improve the performance.
- MPI_Send and MPI_Isend, are **top** in the priority list as MPI decides what is best.
- MPI_Ssend and MPI_Issend, when the sender needs to know **when** the message is received and to avoid local buffering. MPI_Ssend can be useful for debugging.
- MPI_Bsend, MPI_Ibsend, MPI_Rsend, MPI_Irsend are for **fine-tuning** the performance.

- Credits:
  - https://iamsorush.com/posts/mpi-send-types/

Copyright © 2010, Elsevier Inc. All rights Reserved

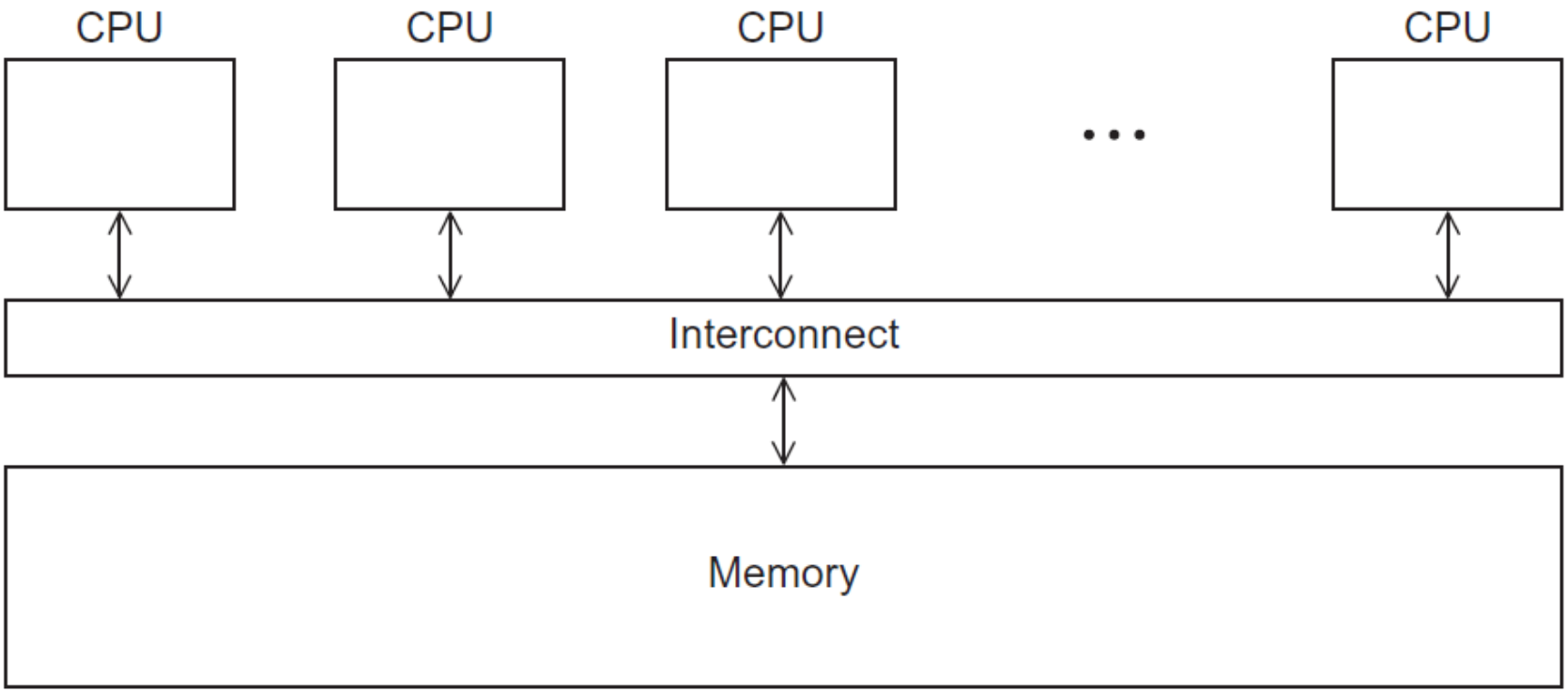
High Performance Computing for Data Science

Shared Memory Programming with OpenMP
Lecture 14 - 30/10/2026

Introduction to OpenMP

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

A shared memory system



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

OpenMP

- An API for shared-memory parallel programming.
- MP = multiprocessing
- Designed for systems in which each thread or process can potentially have access to all available memory.
- System is viewed as a collection of cores or CPU's, all of which have access to main memory.

Copyright © 2010, Elsevier Inc. All rights Reserved

Pragmas

- Special preprocessor instructions.
- Typically added to a system to allow behaviors that aren't part of the basic C specification.
- Compilers that don't support the pragmas ignore them.

#pragma

Copyright © 2010, Elsevier Inc. All rights Reserved

Hello World example

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

 printf("Hello from the serial world\n");

#pragma omp parallel num_threads(10)
 {
  printf("Hello World... from thread = %d\n", omp_get_thread_num());
 }

 printf("Back to the serial world\n");
 return 0;
}
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void);  /* Thread function */

int main(int argc, char* argv[]) {
   /* Get number of threads from command line */
   int thread_count = strtol(argv[1], NULL, 10);

#  pragma omp parallel num_threads(thread_count)
   Hello();

   return 0;
}  /* main */

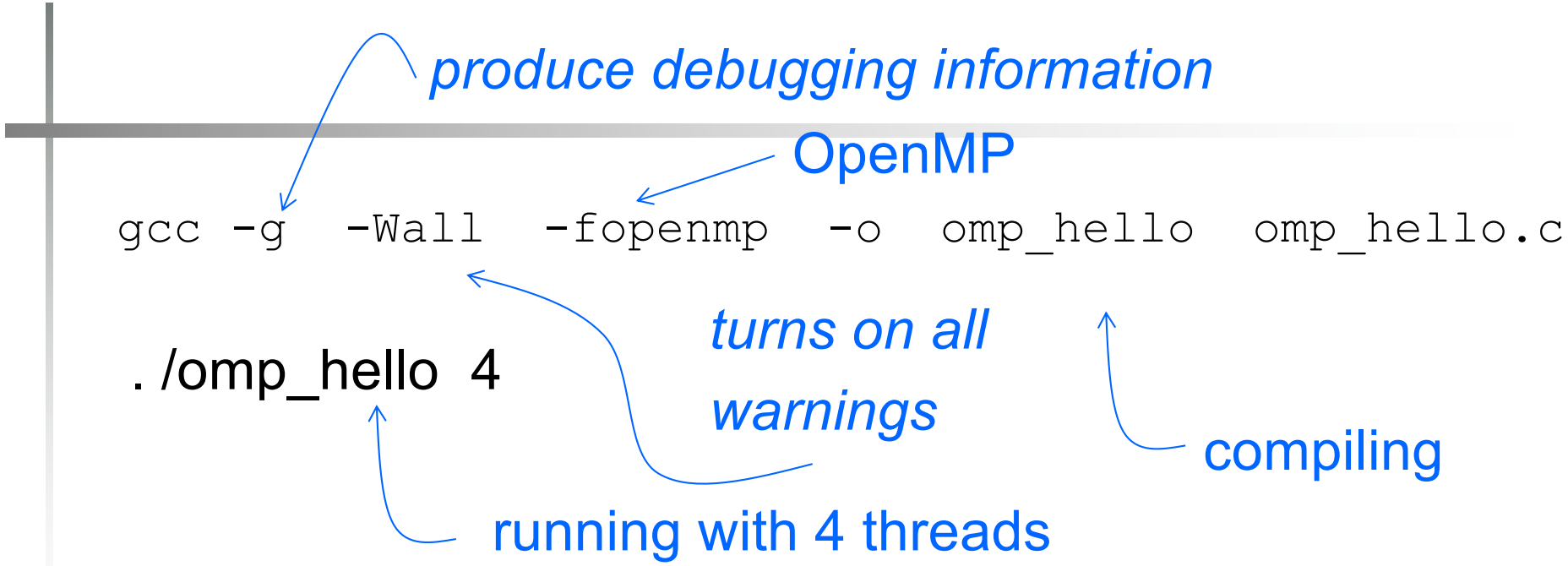
void Hello(void) {
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   printf("Hello from thread %d of %d\n", my_rank, thread_count);

}  /* Hello */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved



Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4

possible outcomes

Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4

Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4

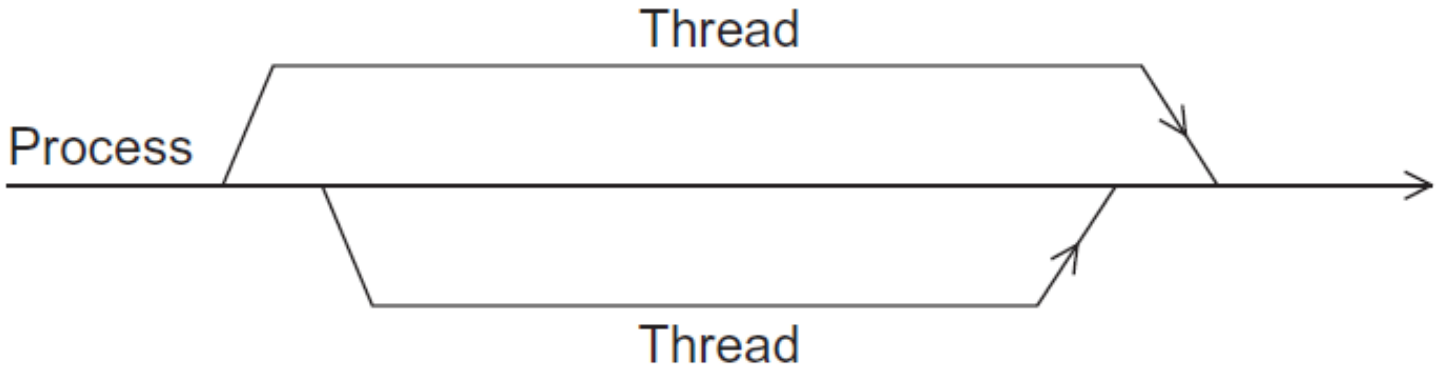
MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

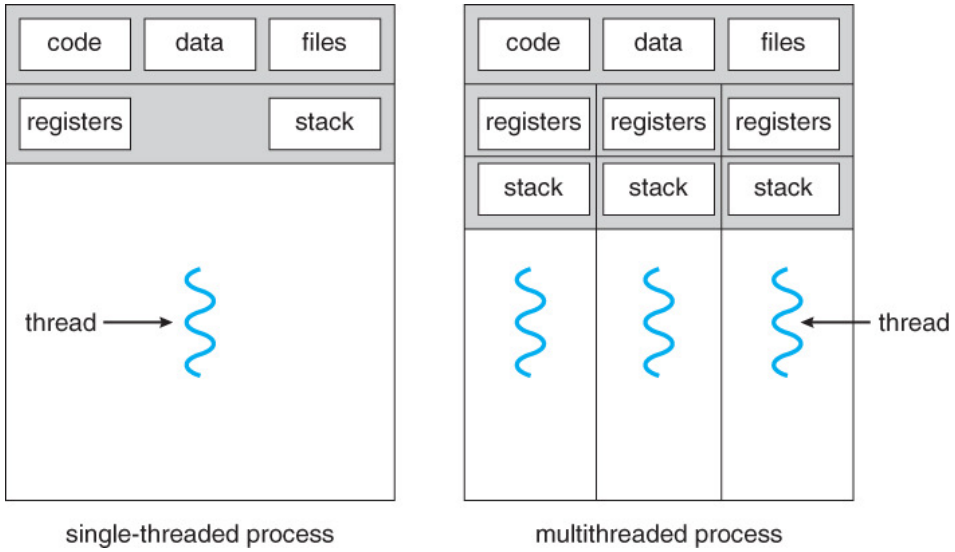
OpenMP pragmas

- # pragma omp parallel
  - Most basic parallel directive.
  - The number of threads that run the following **structured block** of code is determined by the run-time system.

Copyright © 2010, Elsevier Inc. All rights Reserved

A process forking and joining two threads





MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Some terminology

- In OpenMP parlance the collection of threads executing the parallel block — the original thread and the new threads — is called a team, the original thread is called the master, and the additional threads are called slaves.



Copyright © 2010, Elsevier Inc. All rights Reserved

clause

- Text that modifies a directive.
- The num_threads clause can be added to a parallel directive.
- It allows the programmer to specify the number of threads that should execute the following block.

```
# pragma omp parallel num_threads ( thread_count )
```

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

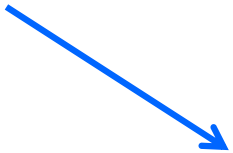
Of note…

- There may be system-defined limitations on the number of threads that a program can start.
- The OpenMP standard doesn't guarantee that this will actually start thread_count threads.
- Most current systems can start hundreds or thousands of threads.
- Unless we're trying to start a lot of threads, we will almost always get the desired number of threads.

Copyright © 2010, Elsevier Inc. All rights Reserved

In case the compiler doesn't support OpenMP

```
# include <omp.h>
```



```
#ifdef _OPENMP
# include <omp.h>
#endif
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

In case the compiler doesn't support OpenMP

```
# ifdef _OPENMP
   int my_rank = omp_get_thread_num ( );
   int thread_count = omp_get_num_threads ( );
# e l s e
   int my_rank = 0;
   int thread_count = 1;
# endif
```


Copyright © 2010, Elsevier Inc. All rights Reserved

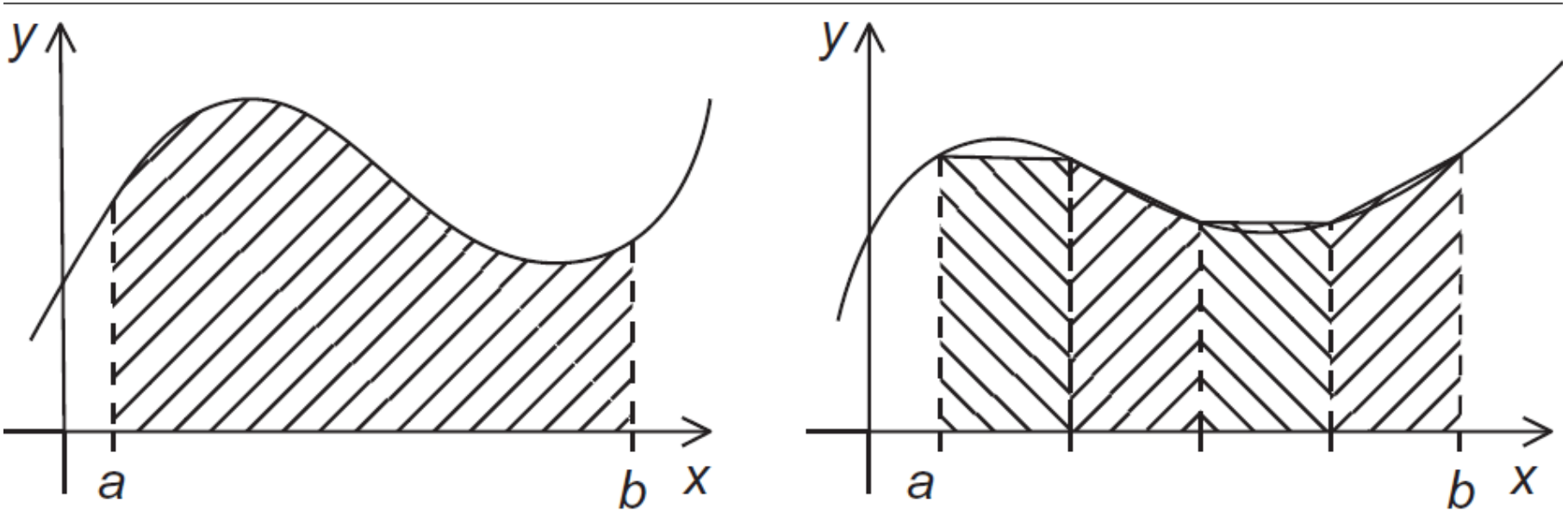
High Performance Computing for Data Science

Shared Memory Programming with OpenMP
Lecture14 - 30/10/2026

The Trapezoidal rule

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

The trapezoidal rule



MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Serial algorithm

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
    x_i = a + i*h;
    approx += f(x_i);
}
approx = h*approx;
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

A First OpenMP Version

1) We identified two types of tasks:
   a) computation of the areas of individual trapezoids, and
   b) adding the areas of trapezoids.
2) There is no communication among the tasks in the first collection, but each task in the first collection communicates with "collecting" task.

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

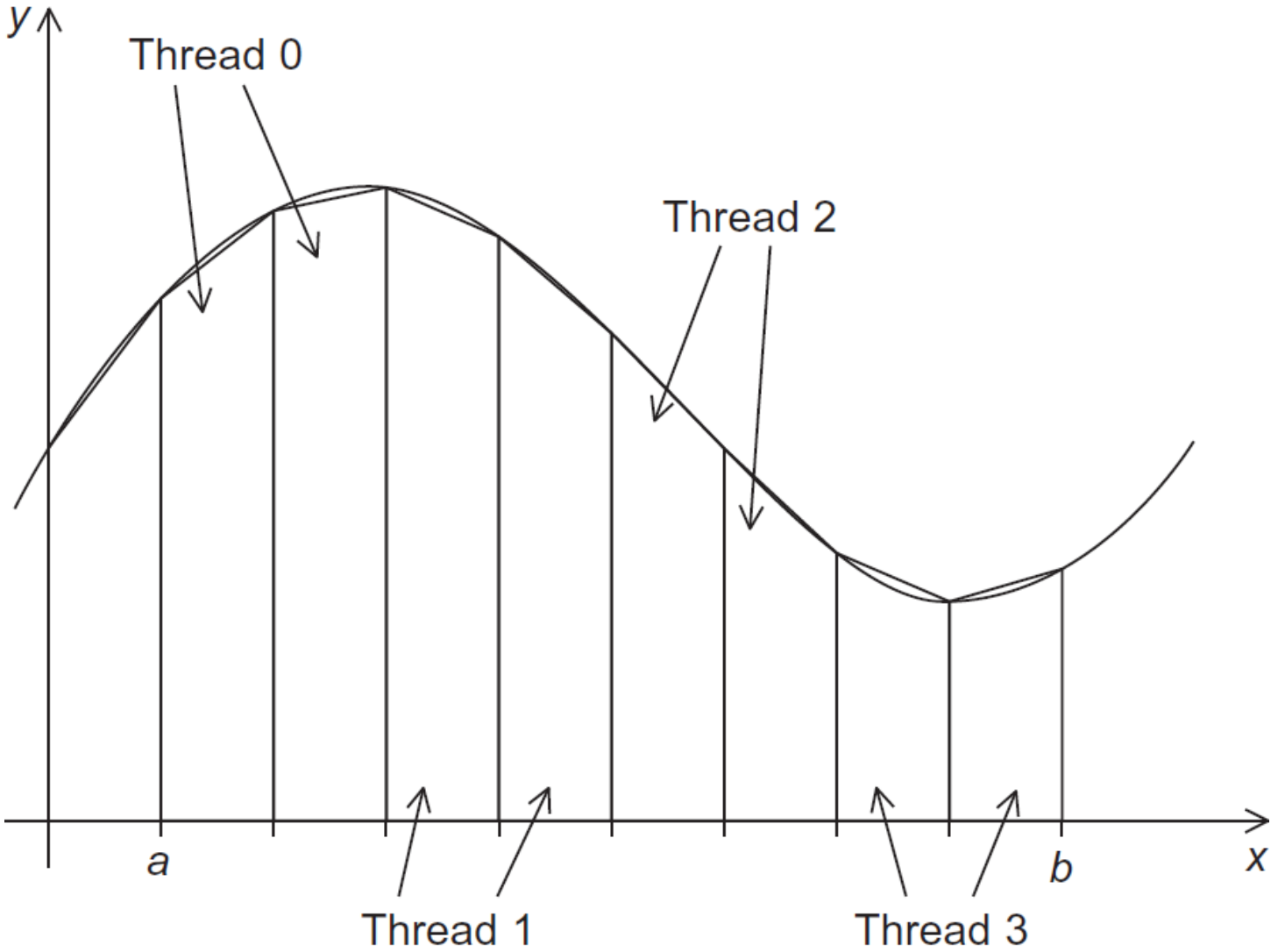
A First OpenMP Version

3) We assumed that there would be many more trapezoids than cores.

- So we aggregated tasks by assigning a contiguous block of trapezoids to each thread (and a single thread to each core).

Copyright © 2010, Elsevier Inc. All rights Reserved

Assignment of trapezoids to threads



MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Time	Thread 0	Thread 1
0	`global_result = 0` to register	finish `my_result`
1	`my_result = 1` to register	`global_result = 0` to register
2	add `my_result` to `global_result`	`my_result = 2` to register
3	store `global_result = 1`	add `my_result` to `global_result`
4		store `global_result = 2`

Unpredictable results when two (or more) threads attempt to simultaneously execute:

global_result += my_result ;

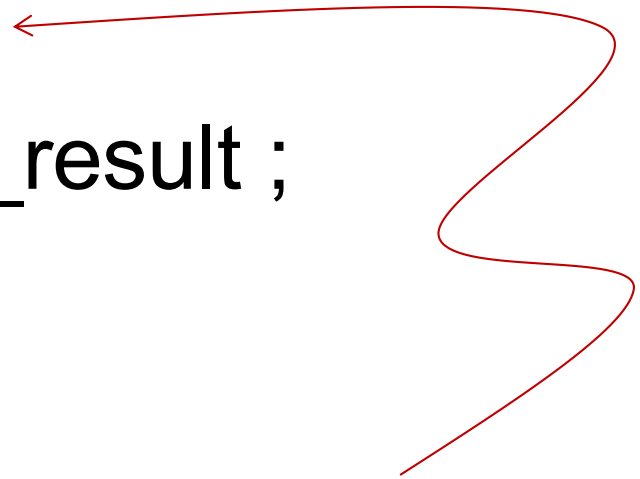


MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Mutual exclusion

```
# pragma omp critical
   global_result += my_result ;
```



only one thread can execute
the following structured block at a time

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
} /* Trap */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved



SCOPE OF VARIABLES

Copyright © 2010, Elsevier Inc. All rights Reserved

Scope

- In serial programming, the **scope** of a variable consists of those **parts of a program in which the variable can be used**.

- In OpenMP, the scope of a variable refers **to the set of threads that can access the variable in a parallel block**.

Copyright © 2010, Elsevier Inc. All rights Reserved

Scope in OpenMP

- A variable that can be accessed by all the threads in the team has shared scope.
- A variable that can only be accessed by a single thread has private scope.
- The default scope for variables declared before a parallel block is shared.



MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
   double  global_result = 0.0;  /* Store result in global_result */
   double  a, b;                 /* Left and right endpoints      */
   int     n;                    /* Total number of trapezoids    */
   int     thread_count;

   thread_count = strtol(argv[1], NULL, 10);
   printf("Enter a, b, and n\n");
   scanf("%lf %lf %d", &a, &b, &n);
#  pragma omp parallel num_threads(thread_count)
   Trap(a, b, n, &global_result);

   printf("With n = %d trapezoids, our estimate\n", n);
   printf("of the integral from %f to %f = %.14e\n",
      a, b, global_result);
   return 0;
}  /* main */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

```
void Trap(double a, double b, int n, double* global_result_p) {
   double  h, x, my_result;
   double  local_a, local_b;
   int  i, local_n;
   int my_rank = omp_get_thread_num();
   int thread_count = omp_get_num_threads();

   h = (b-a)/n;
   local_n = n/thread_count;
   local_a = a + my_rank*local_n*h;
   local_b = local_a + local_n*h;
   my_result = (f(local_a) + f(local_b))/2.0;
   for (i = 1; i <= local_n-1; i++) {
     x = local_a + i*h;
     my_result += f(x);
   }
   my_result = my_result*h;

#  pragma omp critical
   *global_result_p += my_result;
} /* Trap */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Hands-on: OpenMP "Hello World"

```c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

 printf("Hello from the serial world\n");

#pragma omp parallel num_threads(10)
 {
  printf("Hello World... from thread = %d\n", omp_get_thread_num());
 }

 printf("Back to the serial world\n");
 return 0;
}
```

```
gcc -g  -Wall  -fopenmp  -o  omp_hello  omp_hello.c
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Hands-on: MPI Performance benchmark

Matrix Vector multiplication

$$\begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{bmatrix}$$

Copyright © 2010, Elsevier Inc. All rights Reserved

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform